

CO5-AT2-APPLICATION BASED PROBLEM SOLVING

SQL QUERIES :-

```
CREATE DATABASE employee_db;
USE employee_db;
CREATE TABLE department (
    dept_id INT PRIMARY KEY AUTO_INCREMENT,
    dept_name VARCHAR(50) UNIQUE NOT NULL
);
INSERT INTO department (dept_name) VALUES
('CSE'),
('ECE'),
('EEE'),
('IT'),
('MECH'),
('CIVIL'),
('HR'),
('FINANCE');
CREATE TABLE employee (
    employee_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    gender VARCHAR(10) NOT NULL,
    dob DATE NOT NULL,
    contact VARCHAR(15) NOT NULL,
    email VARCHAR(100) NOT NULL,
    address VARCHAR(200),
```

```
dept_id INT NOT NULL,  
designation VARCHAR(50) NOT NULL,  
joining_date DATE NOT NULL,  
basic_salary DOUBLE NOT NULL,  
allowances DOUBLE NOT NULL,  
deductions DOUBLE NOT NULL,  
employment_status VARCHAR(20) NOT NULL,  
FOREIGN KEY (dept_id) REFERENCES department(dept_id)  
);
```

JAVA CODE :-

Class DBConnection.java;

```
import java.sql.Connection;  
import java.sql.DriverManager;  
  
public class DBConnection {  
  
    private static final String URL =  
"jdbc:mysql://localhost:3306/employee_db?useSSL=false&allowPublicKeyRetrieval=true&server  
Timezone=UTC";  
  
    private static final String USER = "studentapp";  
    private static final String PASSWORD = "root";  
  
    public static Connection getConnection() throws Exception {  
  
        Class.forName("com.mysql.cj.jdbc.Driver");  
  
        return DriverManager.getConnection(  
            URL,  
            USER,  
            PASSWORD  
        );  
    }  
}
```

Class EmployeePayroll.java;

```
import java.awt.*;
import java.awt.event.*;
import java.sql.*;

public class EmployeePayroll extends Frame {

    Label lblTitle;
    Label lblSubtitle;

    TextField txtId;
    TextField txtName;
    TextField txtDob;
    TextField txtContact;
    TextField txtEmail;
    TextField txtDesignation;
    TextField txtJoining;
    TextField txtBasic;
    TextField txtAllowances;
    TextField txtDeductions;
    TextField txtGross;
    TextField txtNet;
    TextField txtSearch;

    TextArea txtAddress;
    TextArea txtOutput;

    Choice choiceGender;
    Choice choiceDept;
    Choice choiceStatus;
    Choice choiceSearch;

    Button btnAdd;
    Button btnSearch;
    Button btnUpdate;
    Button btnDelete;
    Button btnClear;
    Button btnView;
    Button btnPayroll;

    Label statusBar;

    Color charcoal = new Color(28, 28, 30);
```

```

Color darkPanel = new Color(42, 42, 45);
Color gold = new Color(198, 164, 92);
Color lightGold = new Color(232, 216, 177);
Color ivory = new Color(247, 244, 236);
Color white = new Color(255, 255, 255);
Color green = new Color(67, 130, 90);
Color red = new Color(170, 65, 65);
Color blue = new Color(65, 91, 120);

public EmployeePayroll() {

    setTitle("SIMATS - Employee Management & Payroll");

    setSize(1200, 800);

    setLayout(new BorderLayout(8, 8));

    setBackground(ivy);

    createInterface();

    setAllFonts(this);

    lblTitle.setFont(
        new Font(
            "Times New Roman",
            Font.BOLD,
            30
        )
    );

    lblSubtitle.setFont(
        new Font(
            "Times New Roman",
            Font.PLAIN,
            14
        )
    );

    createDatabaseTables();

    loadDepartments();

    txtOutput.setText(
        "
            WELCOME TO SIMATS\n\n" +
            "            EMPLOYEE MANAGEMENT & PAYROLL\n\n" +
            "Use the employee profile section to enter details.\n" +

```

```

        "Use the buttons below to manage employee records.\n\n" +
        "Select CALCULATE PAYROLL to calculate salary."

    );

    addWindowListener(
        new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        }
    );

    setLocationRelativeTo(null);

    setVisible(true);
}

void createInterface() {

    Panel header = new Panel(
        new BorderLayout()
    );

    header.setBackground(charcoal);

    lblTitle = new Label(
        "S I M A T S",
        Label.CENTER
    );

    lblTitle.setForeground(lightGold);

    header.add(
        lblTitle,
        BorderLayout.CENTER
    );

    lblSubtitle = new Label(
        "EMPLOYEE MANAGEMENT | PAYROLL | HUMAN RESOURCES",
        Label.CENTER
    );

    lblSubtitle.setForeground(white);

    header.add(
        lblSubtitle,
        BorderLayout.SOUTH
    );
}

```

```

    );

    add(
        header,
        BorderLayout.NORTH
    );

    Panel mainPanel = new Panel(
        new GridLayout(
            1,
            2,
            12,
            0
        )
    );

    mainPanel.setBackground(ivy);

    mainPanel.add(
        createEmployeePanel()
    );

    mainPanel.add(
        createPayrollPanel()
    );

    add(
        mainPanel,
        BorderLayout.CENTER
    );

    add(
        createBottomPanel(),
        BorderLayout.SOUTH
    );
}

Panel createEmployeePanel() {

    Panel outer = new Panel(
        new BorderLayout(
            5,
            5
        )
    );

    outer.setBackground(ivy);

```

```
Label heading = new Label(
    "  EMPLOYEE PROFILE",
    Label.LEFT
);

heading.setFont(
    new Font(
        "Times New Roman",
        Font.BOLD,
        18
    )
);

heading.setForeground(lightGold);

heading.setBackground(charcoal);

outer.add(
    heading,
    BorderLayout.NORTH
);

Panel form = new Panel(
    new GridLayout(
        9,
        2,
        8,
        8
    )
);

form.setBackground(ivory);

txtId = new TextField();

txtName = new TextField();

choiceGender = new Choice();

choiceGender.add("Male");
choiceGender.add("Female");
choiceGender.add("Other");

txtDob = new TextField();

txtContact = new TextField();
```

```
txtEmail = new TextField();

choiceDept = new Choice();

txtDesignation = new TextField();

txtJoining = new TextField();

addLabelField(
    form,
    "Employee ID",
    txtId
);

addLabelField(
    form,
    "Full Name",
    txtName
);

addLabelField(
    form,
    "Gender",
    choiceGender
);

addLabelField(
    form,
    "Date of Birth",
    txtDob
);

addLabelField(
    form,
    "Contact Number",
    txtContact
);

addLabelField(
    form,
    "Email Address",
    txtEmail
);

addLabelField(
    form,
```

```

        "Department",
        choiceDept
    );

    addLabelField(
        form,
        "Designation",
        txtDesignation
    );

    addLabelField(
        form,
        "Joining Date",
        txtJoining
    );

    outer.add(
        form,
        BorderLayout.CENTER
    );

    Panel addressPanel = new Panel(
        new BorderLayout(
            5,
            5
        )
    );

    addressPanel.setBackground(ivory);

    Label addressLabel = new Label(
        "Address"
    );

    addressLabel.setFont(
        new Font(
            "Times New Roman",
            Font.BOLD,
            14
        )
    );

    addressLabel.setForeground(charcoal);

    txtAddress = new TextArea(
        "",
        3,
        40
    );

```

```

    );

    addressPanel.add(
        addressLabel,
        BorderLayout.NORTH
    );

    addressPanel.add(
        txtAddress,
        BorderLayout.CENTER
    );

    outer.add(
        addressPanel,
        BorderLayout.SOUTH
    );

    return outer;
}

Panel createPayrollPanel() {

    Panel outer = new Panel(
        new BorderLayout(
            5,
            8
        )
    );

    outer.setBackground(ivy);

    Label heading = new Label(
        "  COMPENSATION & PAYROLL",
        Label.LEFT
    );

    heading.setFont(
        new Font(
            "Times New Roman",
            Font.BOLD,
            18
        )
    );

    heading.setForeground(lightGold);

    heading.setBackground(charcoal);

```

```
outer.add(
    heading,
    BorderLayout.NORTH
);

Panel salaryPanel = new Panel(
    new GridLayout(
        6,
        2,
        8,
        8
    )
);

salaryPanel.setBackground(ivy);

txtBasic = new TextField();

txtAllowances = new TextField();

txtDeductions = new TextField();

txtGross = new TextField();

txtNet = new TextField();

txtGross.setEditable(false);

txtNet.setEditable(false);

txtGross.setBackground(
    new Color(
        235,
        224,
        194
    )
);

txtNet.setBackground(
    new Color(
        219,
        235,
        222
    )
);
```

```
choiceStatus = new Choice();

choiceStatus.add("Active");
choiceStatus.add("Inactive");
choiceStatus.add("On Leave");

addLabelField(
    salaryPanel,
    "Basic Salary",
    txtBasic
);

addLabelField(
    salaryPanel,
    "Allowances",
    txtAllowances
);

addLabelField(
    salaryPanel,
    "Deductions",
    txtDeductions
);

addLabelField(
    salaryPanel,
    "Gross Salary",
    txtGross
);

addLabelField(
    salaryPanel,
    "Net Salary",
    txtNet
);

addLabelField(
    salaryPanel,
    "Employment Status",
    choiceStatus
);

outer.add(
    salaryPanel,
    BorderLayout.NORTH
);

Panel statementPanel = new Panel(
```

```
        new BorderLayout(
            5,
            5
        )
    );

    statementPanel.setBackground(ivory);

    Label statementTitle = new Label(
        "PAYROLL STATEMENT",
        Label.CENTER
    );

    statementTitle.setFont(
        new Font(
            "Times New Roman",
            Font.BOLD,
            18
        )
    );

    statementTitle.setForeground(gold);

    txtOutput = new TextArea();

    txtOutput.setFont(
        new Font(
            "Times New Roman",
            Font.PLAIN,
            14
        )
    );

    txtOutput.setBackground(
        new Color(
            250,
            249,
            244
        )
    );

    statementPanel.add(
        statementTitle,
        BorderLayout.NORTH
    );

    statementPanel.add(
        txtOutput,
```

```

        BorderLayout.CENTER
    );

    outer.add(
        statementPanel,
        BorderLayout.CENTER
    );

    Panel calculatePanel = new Panel();

    calculatePanel.setBackground(ivy);

    btnPayroll = new Button(
        "CALCULATE PAYROLL"
    );

    styleButton(
        btnPayroll,
        gold
    );

    calculatePanel.add(
        btnPayroll
    );

    outer.add(
        calculatePanel,
        BorderLayout.SOUTH
    );

    btnPayroll.addActionListener(
        e -> calculatePayroll()
    );

    return outer;
}

void addLabelField(
    Panel panel,
    String text,
    Component component) {

    Label label = new Label(
        text
    );

    label.setFont(

```

```

        new Font(
            "Times New Roman",
            Font.BOLD,
            14
        )
    );

    label.setForeground(charcoal);

    panel.add(label);

    component.setBackground(white);

    component.setFont(
        new Font(
            "Times New Roman",
            Font.PLAIN,
            14
        )
    );

    panel.add(component);
}

Panel createBottomPanel() {

    Panel bottom = new Panel(
        new BorderLayout(
            5,
            5
        )
    );

    bottom.setBackground(charcoal);

    Panel buttons = new Panel();

    buttons.setBackground(charcoal);

    btnAdd = new Button(
        "ADD EMPLOYEE"
    );

    btnSearch = new Button(
        "SEARCH"
    );

```

```
btnUpdate = new Button(  
    "UPDATE"  
);  
  
btnDelete = new Button(  
    "DELETE"  
);  
  
btnClear = new Button(  
    "CLEAR"  
);  
  
btnView = new Button(  
    "VIEW ALL"  
);  
  
styleButton(  
    btnAdd,  
    green  
);  
  
styleButton(  
    btnSearch,  
    blue  
);  
  
styleButton(  
    btnUpdate,  
    gold  
);  
  
styleButton(  
    btnDelete,  
    red  
);  
  
styleButton(  
    btnClear,  
    new Color(  
        95,  
        78,  
        65  
    )  
);  
  
styleButton(  
    btnView,  
    new Color(  

```

```

        75,
        75,
        78
    )
);

buttons.add(btnAdd);
buttons.add(btnSearch);
buttons.add(btnUpdate);
buttons.add(btnDelete);
buttons.add(btnClear);
buttons.add(btnView);

bottom.add(
    buttons,
    BorderLayout.NORTH
);

Panel searchPanel = new Panel();

searchPanel.setBackground(charcoal);

Label searchLabel = new Label(
    "Search:"
);

searchLabel.setForeground(white);

choiceSearch = new Choice();

choiceSearch.add(
    "Employee ID"
);

choiceSearch.add(
    "Department"
);

choiceSearch.add(
    "Designation"
);

choiceSearch.add(
    "Status"
);

txtSearch = new TextField(

```

```
18
);

searchPanel.add(
    searchLabel
);

searchPanel.add(
    choiceSearch
);

searchPanel.add(
    txtSearch
);

bottom.add(
    searchPanel,
    BorderLayout.SOUTH
);

statusBar = new Label(
    " Ready"
);

statusBar.setBackground(
    new Color(
        20,
        20,
        22
    )
);

statusBar.setForeground(
    lightGold
);

bottom.add(
    statusBar,
    BorderLayout.CENTER
);

btnAdd.addActionListener(
    e -> addEmployee()
);

btnSearch.addActionListener(
    e -> searchEmployee()
);
```

```

    );

    btnUpdate.addActionListener(
        e -> updateEmployee()
    );

    btnDelete.addActionListener(
        e -> deleteEmployee()
    );

    btnClear.addActionListener(
        e -> clearFields()
    );

    btnView.addActionListener(
        e -> viewAllEmployees()
    );

    return bottom;
}

void styleButton(
    Button button,
    Color color) {

    button.setBackground(color);

    button.setForeground(white);

    button.setFont(
        new Font(
            "Times New Roman",
            Font.BOLD,
            13
        )
    );
}

void setAllFonts(
    Component component) {

    component.setFont(
        new Font(
            "Times New Roman",
            Font.PLAIN,
            14
        )
    );
}

```

```

    );

    if (component instanceof Container) {

        Component[] components =
            ((Container) component).getComponents();

        for (Component c : components) {

            setAllFonts(c);
        }
    }
}

void createDatabaseTables() {

    try {

        Connection con =
            DBConnection.getConnection();

        Statement st =
            con.createStatement();

        st.executeUpdate(
            "CREATE TABLE IF NOT EXISTS department (" +
            "dept_id INT PRIMARY KEY AUTO_INCREMENT," +
            "dept_name VARCHAR(50) UNIQUE NOT NULL)"
        );

        st.executeUpdate(
            "CREATE TABLE IF NOT EXISTS employee (" +
            "employee_id INT PRIMARY KEY," +
            "name VARCHAR(100) NOT NULL," +
            "gender VARCHAR(10) NOT NULL," +
            "dob DATE NOT NULL," +
            "contact VARCHAR(20) NOT NULL," +
            "email VARCHAR(100) NOT NULL," +
            "address VARCHAR(250)," +
            "dept_id INT NOT NULL," +
            "designation VARCHAR(60) NOT NULL," +
            "joining_date DATE NOT NULL," +
            "basic_salary DOUBLE NOT NULL," +
            "allowances DOUBLE NOT NULL," +
            "deductions DOUBLE NOT NULL," +
            "employment_status VARCHAR(20) NOT NULL," +
            "FOREIGN KEY(dept_id) REFERENCES department(dept_id))"
        );
    }
}

```

```

    );

    String[] departments = {
        "CSE",
        "ECE",
        "EEE",
        "IT",
        "MECH",
        "CIVIL",
        "HR",
        "FINANCE"
    };

    for (String dept : departments) {

        PreparedStatement ps =
            con.prepareStatement(
                "INSERT IGNORE INTO department(dept_name) VALUES(?)"
            );

        ps.setString(
            1,
            dept
        );

        ps.executeUpdate();

        ps.close();
    }

    st.close();

    con.close();

} catch (Exception e) {

    if (txtOutput != null) {

        txtOutput.setText(
            "DATABASE ERROR\n\n" +
            e.getMessage() +
            "\n\nCheck MySQL server and DBConnection.java"
        );
    }
}
}

```

```

void loadDepartments() {

    choiceDept.removeAll();

    choiceDept.add("CSE");
    choiceDept.add("ECE");
    choiceDept.add("EEE");
    choiceDept.add("IT");
    choiceDept.add("MECH");
    choiceDept.add("CIVIL");
    choiceDept.add("HR");
    choiceDept.add("FINANCE");

    choiceDept.select("CSE");
}

int getDepartmentId(
    String department)
    throws Exception {

    Connection con =
        DBConnection.getConnection();

    PreparedStatement ps =
        con.prepareStatement(
            "SELECT dept_id FROM department WHERE dept_name=?"
        );

    ps.setString(
        1,
        department
    );

    ResultSet rs =
        ps.executeQuery();

    if (rs.next()) {

        int id =
            rs.getInt(
                "dept_id"
            );

        rs.close();

        ps.close();
    }
}

```

```

        con.close();

        return id;
    }

    rs.close();

    ps.close();

    con.close();

    throw new Exception(
        "Department not found: " +
        department
    );
}

boolean validateInput() {

    if (
        txtId.getText().trim().isEmpty()
        ||
        txtName.getText().trim().isEmpty()
        ||
        txtDob.getText().trim().isEmpty()
        ||
        txtContact.getText().trim().isEmpty()
        ||
        txtEmail.getText().trim().isEmpty()
        ||
        txtDesignation.getText().trim().isEmpty()
        ||
        txtJoining.getText().trim().isEmpty()
        ||
        txtBasic.getText().trim().isEmpty()
        ||
        txtAllowances.getText().trim().isEmpty()
        ||
        txtDeductions.getText().trim().isEmpty()
    ) {

        message(
            "Please fill all required employee details.",
            red
        );

        return false;
    }
}

```

```

try {

    Integer.parseInt(
        txtId.getText().trim()
    );

    Double.parseDouble(
        txtBasic.getText().trim()
    );

    Double.parseDouble(
        txtAllowances.getText().trim()
    );

    Double.parseDouble(
        txtDeductions.getText().trim()
    );

    Date.valueOf(
        txtDob.getText().trim()
    );

    Date.valueOf(
        txtJoining.getText().trim()
    );

} catch (Exception e) {

    message(
        "Invalid input.\n\n" +
        "ID and salary must be numeric.\n" +
        "Date format must be YYYY-MM-DD.",
        red
    );

    return false;
}

return true;
}

void addEmployee() {

    if (!validateInput()) {
        return;
    }
}

```

```

try {

    Connection con =
        DBConnection.getConnection();

    int deptId =
        getDepartmentId(
            choiceDept.getSelectedItem()
        );

    String sql =
        "INSERT INTO employee(" +
            "employee_id,name,gender,dob,contact,email,address," +
            "dept_id,designation,joining_date,basic_salary," +
            "allowances,deductions,employment_status)" +
            " VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?)";

    PreparedStatement ps =
        con.prepareStatement(sql);

    ps.setInt(
        1,
        Integer.parseInt(
            txtId.getText()
        )
    );

    ps.setString(
        2,
        txtName.getText()
    );

    ps.setString(
        3,
        choiceGender.getSelectedItem()
    );

    ps.setDate(
        4,
        Date.valueOf(
            txtDob.getText()
        )
    );

    ps.setString(
        5,

```

```
        txtContact.getText()
    );

    ps.setString(
        6,
        txtEmail.getText()
    );

    ps.setString(
        7,
        txtAddress.getText()
    );

    ps.setInt(
        8,
        deptId
    );

    ps.setString(
        9,
        txtDesignation.getText()
    );

    ps.setDate(
        10,
        Date.valueOf(
            txtJoining.getText()
        )
    );

    ps.setDouble(
        11,
        Double.parseDouble(
            txtBasic.getText()
        )
    );

    ps.setDouble(
        12,
        Double.parseDouble(
            txtAllowances.getText()
        )
    );

    ps.setDouble(
        13,
        Double.parseDouble(
            txtDeductions.getText()
        )
    );
```

```

        )
    );

    ps.setString(
        14,
        choiceStatus.getSelectedItem()
    );

    ps.executeUpdate();

    ps.close();

    con.close();

    calculatePayroll();

    message(
        "Employee added successfully.",
        green
    );

} catch (SQLIntegrityConstraintViolationException e) {

    message(
        "Employee ID already exists.",
        red
    );

} catch (Exception e) {

    message(
        "ADD ERROR\n\n" +
        e.getMessage(),
        red
    );
}
}

void searchEmployee() {

    String value =
        txtSearch.getText().trim();

    if (value.isEmpty()) {

        message(

```

```

        "Enter a search value.",
        red
    );

    return;
}

try {

    Connection con =
        DBConnection.getConnection();

    String type =
        choiceSearch.getSelectedItemAt();

    String sql;

    if (
        type.equals(
            "Employee ID"
        )
    ) {

        sql =
            "SELECT e.*,d.dept_name " +
            "FROM employee e " +
            "JOIN department d " +
            "ON e.dept_id=d.dept_id " +
            "WHERE e.employee_id=?";

    } else if (
        type.equals(
            "Department"
        )
    ) {

        sql =
            "SELECT e.*,d.dept_name " +
            "FROM employee e " +
            "JOIN department d " +
            "ON e.dept_id=d.dept_id " +
            "WHERE d.dept_name=?";

    } else if (
        type.equals(
            "Designation"
        )
    )

```

```

    ) {

        sql =
            "SELECT e.*,d.dept_name " +
            "FROM employee e " +
            "JOIN department d " +
            "ON e.dept_id=d.dept_id " +
            "WHERE e.designation=?";

    } else {

        sql =
            "SELECT e.*,d.dept_name " +
            "FROM employee e " +
            "JOIN department d " +
            "ON e.dept_id=d.dept_id " +
            "WHERE e.employment_status=?";

    }

    PreparedStatement ps =
        con.prepareStatement(sql);

    if (
        type.equals(
            "Employee ID"
        )
    ) {

        ps.setInt(
            1,
            Integer.parseInt(value)
        );

    } else {

        ps.setString(
            1,
            value
        );

    }

    ResultSet rs =
        ps.executeQuery();

    if (!rs.next()) {

        message(

```

```

        "Employee not found.",
        red
    );

} else {

    if (
        type.equals(
            "Employee ID"
        )
    ) {

        fillFields(rs);

        calculatePayroll();

        message(
            "Employee found successfully.",
            green
        );

    } else {

        txtOutput.setText(
            "
                SEARCH RESULTS\n\n"
        );

        txtOutput.append(
            String.format(
                "%-6s %-18s %-12s %-18s %-12s\n",
                "ID",
                "NAME",
                "DEPT",
                "DESIGNATION",
                "STATUS"
            )
        );

        txtOutput.append(
            "-----\n"
        );

        do {

            txtOutput.append(
                String.format(

```

```

        "%-6d %-18s %-12s %-18s %-12s\n",
        rs.getInt(
            "employee_id"
        ),
        rs.getString(
            "name"
        ),
        rs.getString(
            "dept_name"
        ),
        rs.getString(
            "designation"
        ),
        rs.getString(
            "employment_status"
        )
    );

    } while (rs.next());

    statusLabel(
        "Search completed",
        green
    );
}

rs.close();

ps.close();

con.close();

} catch (Exception e) {

    message(
        "SEARCH ERROR\n\n" +
        e.getMessage(),
        red
    );
}

}

void fillFields(
    ResultSet rs)
    throws Exception {

```

```
txtId.setText(  
    String.valueOf(  
        rs.getInt(  
            "employee_id"  
        )  
    )  
);
```

```
txtName.setText(  
    rs.getString(  
        "name"  
    )  
);
```

```
choiceGender.select(  
    rs.getString(  
        "gender"  
    )  
);
```

```
txtDob.setText(  
    rs.getDate(  
        "dob"  
    ).toString()  
);
```

```
txtContact.setText(  
    rs.getString(  
        "contact"  
    )  
);
```

```
txtEmail.setText(  
    rs.getString(  
        "email"  
    )  
);
```

```
txtAddress.setText(  
    rs.getString(  
        "address"  
    )  
);
```

```
choiceDept.select(  
    rs.getString(  
        "dept_name"  
    )  
);
```

```

    );

    txtDesignation.setText(
        rs.getString(
            "designation"
        )
    );

    txtJoining.setText(
        rs.getDate(
            "joining_date"
        ).toString()
    );

    txtBasic.setText(
        String.valueOf(
            rs.getDouble(
                "basic_salary"
            )
        )
    );

    txtAllowances.setText(
        String.valueOf(
            rs.getDouble(
                "allowances"
            )
        )
    );

    txtDeductions.setText(
        String.valueOf(
            rs.getDouble(
                "deductions"
            )
        )
    );

    choiceStatus.select(
        rs.getString(
            "employment_status"
        )
    );
}

void updateEmployee() {

    if (!validateInput()) {

```

```

        return;
    }

    try {

        Connection con =
            DBConnection.getConnection();

        int deptId =
            getDepartmentId(
                choiceDept.getSelectedItem()
            );

        String sql =
            "UPDATE employee SET " +
                "name=?,gender=?,dob=?,contact=?,email=?" +
                "address=?,dept_id=?,designation=?,joining_date=?" +
                "basic_salary=?,allowances=?,deductions=?" +
                "employment_status=? " +
                "WHERE employee_id=?";

        PreparedStatement ps =
            con.prepareStatement(sql);

        ps.setString(
            1,
            txtName.getText()
        );

        ps.setString(
            2,
            choiceGender.getSelectedItem()
        );

        ps.setDate(
            3,
            Date.valueOf(
                txtDob.getText()
            )
        );

        ps.setString(
            4,
            txtContact.getText()
        );

        ps.setString(

```

```
        5,
        txtEmail.getText()
    );

    ps.setString(
        6,
        txtAddress.getText()
    );

    ps.setInt(
        7,
        deptId
    );

    ps.setString(
        8,
        txtDesignation.getText()
    );

    ps.setDate(
        9,
        Date.valueOf(
            txtJoining.getText()
        )
    );

    ps.setDouble(
        10,
        Double.parseDouble(
            txtBasic.getText()
        )
    );

    ps.setDouble(
        11,
        Double.parseDouble(
            txtAllowances.getText()
        )
    );

    ps.setDouble(
        12,
        Double.parseDouble(
            txtDeductions.getText()
        )
    );

    ps.setString(
```

```

        13,
        choiceStatus.getSelectedItemAt()
    );

    ps.setInt(
        14,
        Integer.parseInt(
            txtId.getText()
        )
    );

    int rows =
        ps.executeUpdate();

    ps.close();

    con.close();

    if (rows > 0) {

        calculatePayroll();

        message(
            "Employee updated successfully.",
            green
        );

    } else {

        message(
            "Employee not found.",
            red
        );

    }

} catch (Exception e) {

    message(
        "UPDATE ERROR\n\n" +
        e.getMessage(),
        red
    );

}

}

void deleteEmployee() {

```

```
if (
    txtId.getText()
        .trim()
        .isEmpty()
) {

    message(
        "Enter Employee ID.",
        red
    );

    return;
}

try {

    Connection con =
        DBConnection.getConnection();

    PreparedStatement ps =
        con.prepareStatement(
            "DELETE FROM employee WHERE employee_id=?"
        );

    ps.setInt(
        1,
        Integer.parseInt(
            txtId.getText()
        )
    );

    int rows =
        ps.executeUpdate();

    ps.close();

    con.close();

    if (rows > 0) {

        clearFields();

        message(
            "Employee deleted successfully.",
            green
        );
    }
}
```

```

    } else {

        message(
            "Employee not found.",
            red
        );
    }

} catch (Exception e) {

    message(
        "DELETE ERROR\n\n" +
        e.getMessage(),
        red
    );
}

}

void viewAllEmployees() {

    try {

        Connection con =
            DBConnection.getConnection();

        Statement st =
            con.createStatement();

        ResultSet rs =
            st.executeQuery(
                "SELECT e.employee_id,e.name," +
                "d.dept_name,e.designation," +
                "e.basic_salary,e.allowances," +
                "e.employment_status " +
                "FROM employee e " +
                "JOIN department d " +
                "ON e.dept_id=d.dept_id " +
                "ORDER BY e.employee_id"
            );

        txtOutput.setText(
            "
                EMPLOYEE DIRECTORY\n\n"
        );

        txtOutput.append(
            String.format(
                "%-6s %-18s %-12s %-18s %-14s %-12s\n",

```

```

        "ID",
        "NAME",
        "DEPT",
        "DESIGNATION",
        "GROSS",
        "STATUS"
    )
);

txtOutput.append(
    "-----\n"
);

int count = 0;

while (rs.next()) {

    double gross =
        rs.getDouble(
            "basic_salary"
        )
        +
        rs.getDouble(
            "allowances"
        );

    txtOutput.append(
        String.format(
            "%-6d %-18s %-12s %-18s %-14.2f %-12s\n",
            rs.getInt(
                "employee_id"
            ),
            rs.getString(
                "name"
            ),
            rs.getString(
                "dept_name"
            ),
            rs.getString(
                "designation"
            ),
            gross,
            rs.getString(
                "employment_status"
            )
        )
    );
};

```

```

        count++;
    }

    txtOutput.append(
        "\nTotal Employees: " +
        count
    );

    rs.close();

    st.close();

    con.close();

    statusLabel(
        "Employee directory loaded",
        green
    );
} catch (Exception e) {

    message(
        "VIEW ERROR\n\n" +
        e.getMessage(),
        red
    );
}

}

void calculatePayroll() {

    try {

        double basic =
            Double.parseDouble(
                txtBasic.getText()
            );

        double allowances =
            Double.parseDouble(
                txtAllowances.getText()
            );

        double deductions =
            Double.parseDouble(
                txtDeductions.getText()
            );
    }
}

```

```

    );

    double gross =
        basic + allowances;

    double net =
        gross - deductions;

    txtGross.setText(
        String.format(
            "%.2f",
            gross
        )
    );

    txtNet.setText(
        String.format(
            "%.2f",
            net
        )
    );

    txtOutput.setText(
        "
            SIMATS PAYROLL STATEMENT\n\n" +
            "Employee ID      : " +
            txtId.getText() +
            "\nEmployee Name    : " +
            txtName.getText() +
            "\nDepartment         : " +
            choiceDept.getSelectedItem() +
            "\nDesignation         : " +
            txtDesignation.getText() +
            "\nEmployment Status : " +
            choiceStatus.getSelectedItem() +
            "\n\n" +
            "-----\n" +
            "Basic Salary      : INR " +
            String.format(
                "%.2f",
                basic
            ) +
            "\nAllowances        : INR " +
            String.format(
                "%.2f",
                allowances
            ) +
            "\n-----\n" +
            "Gross Salary      : INR " +
            String.format(

```

```

                "%.2f",
                gross
            ) +
            "\nDeductions          : INR " +
            String.format(
                "%.2f",
                deductions
            ) +
            "\n-----\n" +
            "NET SALARY          : INR " +
            String.format(
                "%.2f",
                net
            ) +
            "\n-----"
        );

        statusLabel(
            "Payroll calculated successfully",
            green
        );
    } catch (Exception e) {

        message(
            "Enter valid salary values.",
            red
        );
    }
}

void clearFields() {

    txtId.setText("");

    txtName.setText("");

    txtDob.setText("");

    txtContact.setText("");

    txtEmail.setText("");

    txtAddress.setText("");

    txtDesignation.setText("");

```

```

txtJoining.setText("");

txtBasic.setText("");

txtAllowances.setText("");

txtDeductions.setText("");

txtGross.setText("");

txtNet.setText("");

txtSearch.setText("");

choiceGender.select(0);

choiceDept.select("CSE");

choiceStatus.select("Active");

txtOutput.setText(
    "
        WELCOME TO SIMATS\n\n" +
        "        EMPLOYEE MANAGEMENT & PAYROLL\n\n" +
        "Enter employee details and select an operation."
);

statusLabel(
    "Ready",
    green
);
}

void message(
    String text,
    Color color) {

    if (txtOutput != null) {

        txtOutput.setText(text);
    }

    statusLabel(
        "System message",
        color
    );
}

```

```
void statusLabel(  
    String text,  
    Color color) {  
  
    if (statusBar != null) {  
  
        statusBar.setText(  
            " " + text  
        );  
  
        statusBar.setForeground(  
            color  
        );  
    }  
}  
  
public static void main(  
    String[] args) {  
  
    new EmployeePayroll();  
}  
}
```

OUTPUT :-

SIMATS - Employee Management & Payroll

SIMATS

EMPLOYEE MANAGEMENT | PAYROLL | HUMAN RESOURCES

EMPLOYEE PROFILE

Employee ID

101

Full Name

Hema

Gender

Female

Date of Birth

2007-10-05

Contact Number

7358287391

Email Address

hemapriya5107@gmail.com

Department

IT

Designation

Professor

Joining Date

2022-10-05

Address

Porur,Chennai-600116

Basic Salary

50000

Allowances

20000

Deductions

5000

Gross Salary

70000.00

Net Salary

65000.00

Employment Status

Active

PAYROLL STATEMENT

Employee added successfully.

CALCULATE PAYROLL

ADD EMPLOYEE

SEARCH

UPDATE

DELETE

CLEAR

VIEW ALL

System message

Search: Employee ID

SIMATS - Employee Management & Payroll

SIMATS

EMPLOYEE MANAGEMENT | PAYROLL | HUMAN RESOURCES

EMPLOYEE PROFILE

Employee ID

101

Full Name

Hema

Gender

Female

Date of Birth

2007-10-05

Contact Number

7358287391

Email Address

hemapriya5107@gmail.com

Department

IT

Designation

Professor

Joining Date

2022-10-05

Address

Porur,Chennai-600116

Basic Salary

50000

Allowances

20000

Deductions

5000

Gross Salary

70000.00

Net Salary

65000.00

Employment Status

Active

PAYROLL STATEMENT

Employee updated successfully.

CALCULATE PAYROLL

ADD EMPLOYEE

SEARCH

UPDATE

DELETE

CLEAR

VIEW ALL

System message

Search: Employee ID

SIMATS - Employee Management & Payroll

SIMATS

EMPLOYEE MANAGEMENT | PAYROLL | HUMAN RESOURCES

EMPLOYEE PROFILE

Employee ID

101

Full Name

Hema

Gender

Female

Date of Birth

2007-10-05

Contact Number

7358287391

Email Address

hemapriya5107@gmail.com

Department

IT

Designation

Professor

Joining Date

2022-10-05

Address

Porur,Chennai-600116

Basic Salary

50000.0

Allowances

20000.0

Deductions

5000.0

Gross Salary

70000.00

Net Salary

65000.00

Employment Status

Active

PAYROLL STATEMENT

Employee found successfully.

CALCULATE PAYROLL

ADD EMPLOYEE

SEARCH

UPDATE

DELETE

CLEAR

VIEW ALL

System message

Search: Employee ID 101

SIMATS - Employee Management & Payroll

SIMATS

EMPLOYEE MANAGEMENT | PAYROLL | HUMAN RESOURCES

EMPLOYEE PROFILE

Employee ID

101

Full Name

Hema

Gender

Female

Date of Birth

2007-10-05

Contact Number

7358287391

Email Address

hemapriya5107@gmail.com

Department

IT

Designation

Professor

Joining Date

2022-10-05

Address

Porur,Chennai-600116

Basic Salary

50000.0

Allowances

20000.0

Deductions

5000.0

Gross Salary

70000.00

Net Salary

65000.00

Employment Status

Active

PAYROLL STATEMENT

EMPLOYEE DIRECTORY

ID	NAME	DEPT	DESIGNATION	GROSS	STATUS
101	Hema	IT	Professor	70000.00	Active

Total Employees: 1

CALCULATE PAYROLL

ADD EMPLOYEE

SEARCH

UPDATE

DELETE

CLEAR

VIEW ALL

Employee directory loaded

Search: Employee ID 101